

BOTICS ALPHA

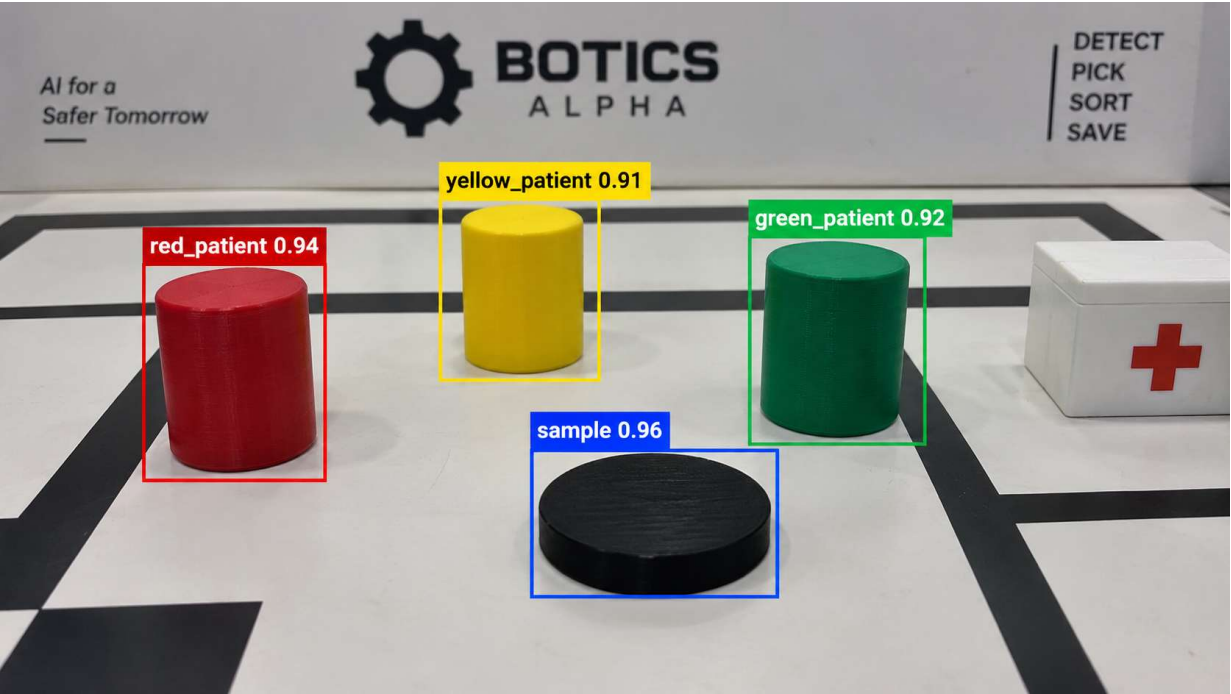
AI Vision Pipeline

Project Report and Technical Documentation

Camera-based multi-class detection for autonomous pick, sort and place, built on the Grove Vision AI V2 with on-device NPU inference.

Item	Detail
Prepared for	BOTICS ALPHA
Project theme	AI for a Safer Tomorrow – Detect · Pick · Sort · Save
Vision board	Seeed Grove Vision AI V2 (Ethos-U55 NPU)
Model	YOLOv8n, 4 classes, INT8, Vela-compiled
Date	September 2026

How this document is organised
Sections 1–5 explain **what was built and why** (project story, architecture, model). Sections 6–8 are the **technical reference** (protocol, code, how to run). Sections 9–13 cover **issues, status, remaining work and deliverables**. Section 14 is a quick-reference sheet.



Contents

1. Executive Summary
2. Project Overview and Architecture
3. Hardware and Software
4. Development Journey (Phases 1–4)
5. Model Specification and Metrics
6. GV2 Serial Protocol
7. Software Tools
8. Setup and Run Guide
9. Issues and Resolutions
10. Current Project Status
11. Open Items to Verify
12. Remaining Work
13. Deliverables
14. Quick Reference

1. Executive Summary

BOTICS ALPHA needed an on-device AI vision pipeline that detects four fixed object classes (`red_patient`, `yellow_patient`, `green_patient`, `sample`) and passes their positions to a robotic arm. All live inference must run on the Grove Vision AI V2 NPU at competition time; the PC is used only for debugging and calibration.

The first model, based on YOLO26, compiled and ran on the NPU but always returned empty detections. Sseed confirmed that the GV2 firmware does not support YOLO26 metadata. The model was therefore rebuilt on YOLOv8n, an architecture Sseed officially supports, trained on recovered real data, exported with a compatible toolchain and compiled with 100% NPU coverage. On the first live test it detected all four classes at 89–97% confidence.

Result	Value
Model architecture / input	YOLOv8n, 192 × 192, INT8, NHWC
Training data	400 real labelled images (100 per class), 340 train / 60 validation
Validation precision / recall	0.993 / 1.0
Validation mAP50 / mAP50-95	0.995 / 0.964
NPU operator coverage	100% (0 CPU-fallback ops)
Offline confidence test	60 / 60 correct class detections
Live hardware test	All 4 classes detected simultaneously, 89–97% confidence
On-device speed	About 7 ms preprocess + 88 ms inference (roughly 10 FPS)

Still to do: pixel-to-robot calibration with real points, hardware test of the XIAO → Mega link, arm/motor control logic, and a full end-to-end field test in the competition scene (see Section 12).

2. Project Overview and Architecture

The arena contains three coloured "patient" cylinders (red, yellow, green) and a black disc called "sample". The arena banner reads **DETECT – PICK – SORT – SAVE**. The robot arm must identify each object with its camera, pick it and sort it to the correct place. Figure 1 shows the trained model on a reference image of the arena.

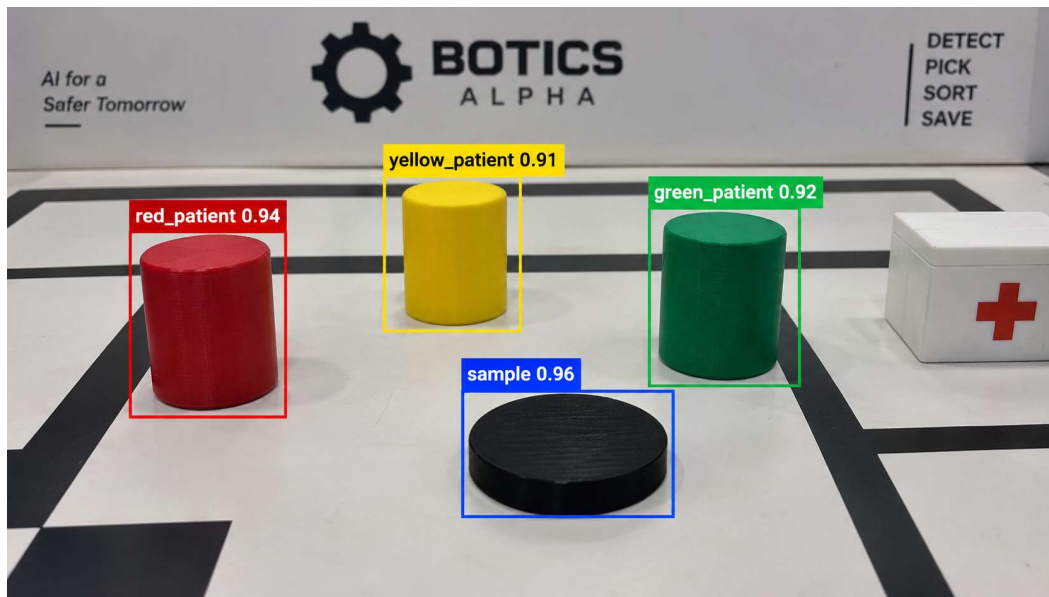


Figure 1: Reference arena image – all four classes detected (0.91 – 0.96 confidence)

2.1 Classes

target_id	Class name	Meaning	Overlay colour
0	red_patient	Red cylinder	Red
1	yellow_patient	Yellow cylinder	Yellow
2	green_patient	Green cylinder	Green
3	sample	Black disc	Dark grey

This mapping is fixed and identical in every script (`CLASS_NAMES`).

2.2 System architecture

Target (competition-time) architecture: **OV5647 camera** → **Grove Vision AI V2** → **XIAO (serial bridge)** → **Arduino Mega 2560** → **coordinate mapping** → **robotic arm**. During development, a PC sits in the loop for visualisation, calibration and testing (dashed path in Figure 2).

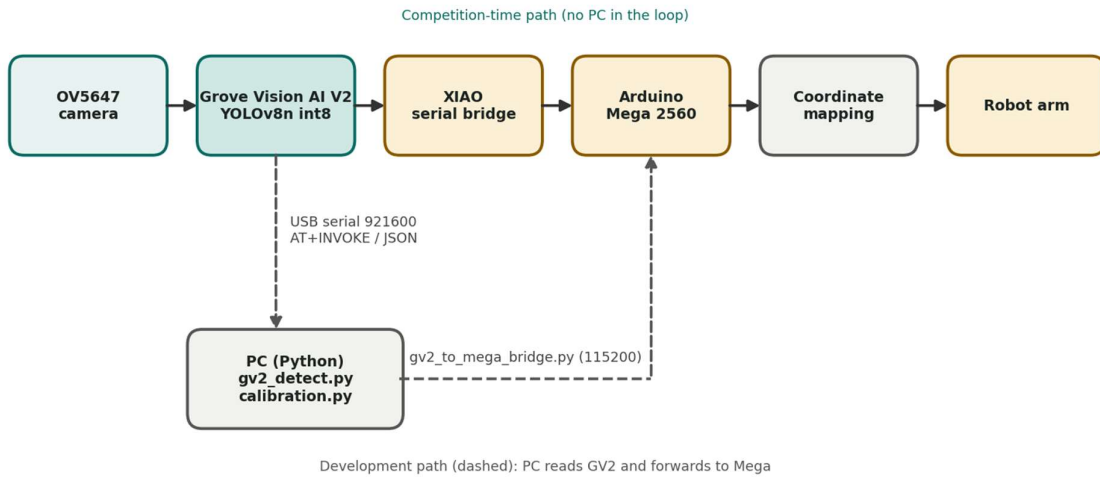


Figure 2: System architecture – competition path (solid) and development path (dashed)

Design rule: no PC inference

All inference runs on the GV2 NPU. The PC never loads `best.pt` and never runs YOLO. PC-side scripts only read the device output over serial, draw it, log it and calibrate against it.

Why a bridge is needed

Raw SSCMA JSON is impractical to parse on an 8-bit AVR Mega (very little RAM). For the competition, an on-device XIAO bridge is used. During development, `gv2_to_mega_bridge.py` on the PC plays the same role so the Mega firmware can be tested today.

3. Hardware and Software

Item	Detail
Camera	OV5647, mounted on the robot arm (eye-in-hand)
Vision board	Seeed Grove Vision AI V2 (GV2), Ethos-U55 NPU; appeared on COM13
Serial bridge	XIAO (competition-time GV2 → Mega link)
Controller	Arduino Mega 2560 (development port e.g. COM7, 115200 baud)
PC / IDE	Windows, VS Code, Python 3.14.3
Python libraries	<code>pyserial</code> , <code>opencv-python</code> , <code>numpy</code>
Flashing / live test	SenseCraft AI (web)
Firmware protocol	SSCMA-Micro AT protocol (verified against the SSCMA-Micro documentation)
Training / export	Ultralytics YOLOv8n → INT8 TFLite → Vela compiler

4. Development Journey (Phases 1–4)

4.1 Phase 1 – Collect all Data

In Phase 1, we will collect approximately 100 images for each patient using the Grove Vision AI Module V2 (GV2) camera. These images will be used to create a reliable dataset for further analysis and model training. The collected images will then be labelled and annotated using SenseCraft AI, ensuring that each image is correctly categorized according to the required parameters. This phase will provide the foundation for developing and training the AI model in the later stages.

4.2 Phase 2 – Building on a supported architecture

Real training data. The original image folder had no usable labels. Four separately trained SenseCraft single-class exports each contained an `annotations.json` with pixel-accurate boxes from earlier SenseCraft sessions. A conversion script merged them into one dataset of **400 images (100 per class)** with a **340 / 60** train/validation split.

Training. A YOLOv8n model, confirmed in Seeed's documentation as supported on Grove Vision AI V2, was trained at the documented input size of **192 × 192** (the old architecture used 224 × 224).

Export and NPU compilation. The installed Ultralytics version defaulted to a newer export backend that produced an incompatible format (NCHW, float32 I/O). A verified older export path restored the required **INT8, NHWC** format. The Vela compile then ran with **100% of operations on the NPU and zero CPU-fallback operations**.

Offline validation before touching hardware. All 60 held-out images were run through both the PyTorch model and the exact INT8 arithmetic of the NPU-compiled file. All 60 produced the correct class detection, against the previous model's persistent zero detections.

4.3 Phase 3 – Hardware deployment and validation

The new model was uploaded to the GV2 through SenseCraft. On the very first live test the device detected all four classes at the same time with confidence between 89% and 97%, which confirmed the root-cause diagnosis and closed the core model problem. Figure 3 shows the SenseCraft live view with the raw Device Logger output.

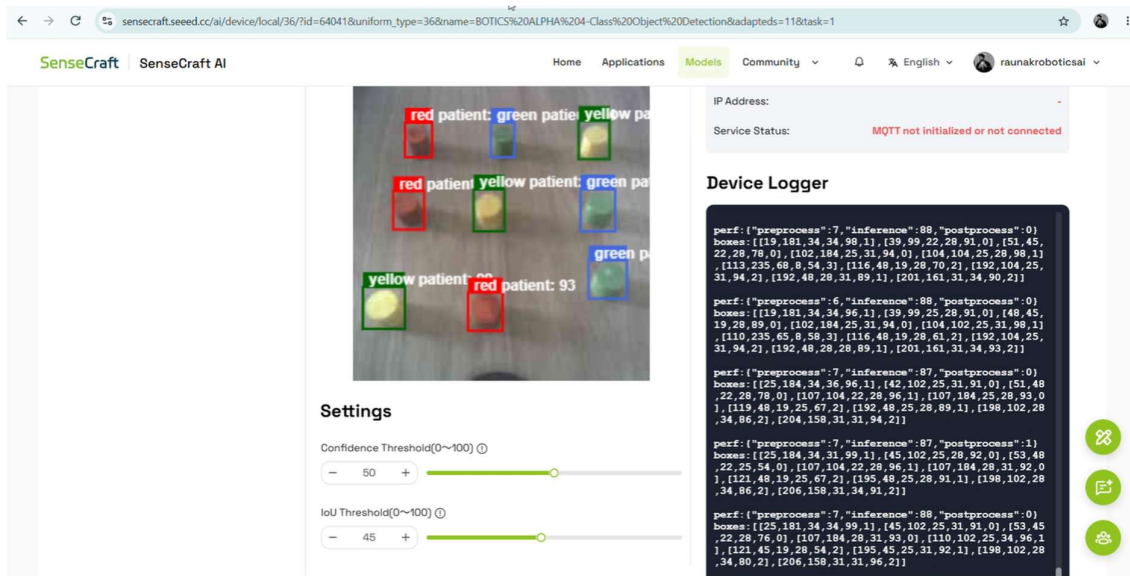


Figure 3: SenseCraft AI – live detection, Confidence 50 / IoU 45 and the raw Device Logger

```
perf:{"preprocess":7,"inference":88,"postprocess":0}
boxes: [[19,181,34,34,98,1],[39,99,22,28,91,0],[51,45,22,28,78,0],
        [102,184,25,31,94,0],[104,104,25,28,98,1],[113,235,68,8,54,3],
        [116,48,19,28,70,2],[192,104,25,31,94,2],[192,48,28,31,89,1],
        [201,161,31,34,90,2]]
```

Observation	Meaning
Confidence 50, IoU 45	SenseCraft settings; Python default threshold is also 0.5
preprocess $\approx$ 7 ms, inference $\approx$ 88 ms	About 100 ms per frame, roughly 10 FPS on-device
Box format [x, y, w, h, score, target_id]	score is an integer 0–100, target_id is 0–3
9–10 objects in one frame	Multiple detections arrive together; the code handles all of them

4.4 Phase 4 – PC-side visualisation and coordinate fix

gv2_detect.py was written to connect to the device over its debug serial link, show detections live and print each detection's class, confidence, position and centre for calibration and debugging. Figure 4 shows it running.

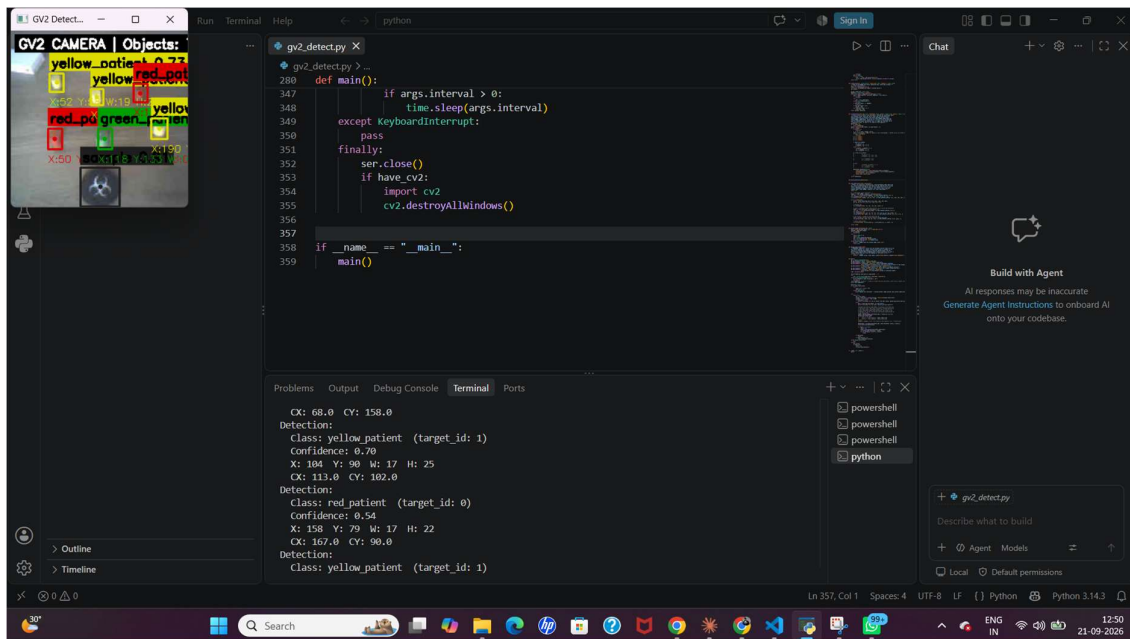


Figure 4: gv2_detect.py – live OpenCV window (top-left) and terminal output

```
Detection:
Class: yellow_patient (target_id: 1)
Confidence: 0.70
X: 104 Y: 90 W: 17 H: 25
CX: 113.0 CY: 102.0
```

Issue: bounding boxes appeared offset from the real objects. It was root-caused from real device data as two compounding problems:

1. **Coordinate space:** box coordinates needed to be matched to the actual preview image, whose real size was measured at **240 × 240** (square) using a temporary `debug_frame_shape()` helper.
2. **Centre convention:** the device reports each box's (x, y) as its **centre point** (standard for this model family), not its top-left corner. The remaining shift was about half a box width and height, which pointed to this.

The fix added the flag `BOX_XY_IS_CENTER = True` and the conversion below. Boxes and centre markers were then checked against live hardware on several classes.

```
if BOX_XY_IS_CENTER:
    cx, cy = x, y
    x_topleft = cx - w/2
    y_topleft = cy - h/2
else:
    x_topleft, y_topleft = x, y
    cx = x_topleft + w/2
    cy = y_topleft + h/2
```

If a different firmware version reports top-left instead, set the flag to `False` and re-test. See also Section 11 for a check on the scaling step.

5. Model Specification and Metrics

Property	Value
Model file	best_full_integer_quant_vela.tflite (NPU-ready)
Architecture	YOLOv8n (models_v2_yolov8n)
Input	192 × 192, NHWC, INT8 (full integer quantisation)
Classes	red_patient, yellow_patient, green_patient, sample
Training images	400 real labelled images (100 per class); 340 train / 60 validation
Validation precision / recall	0.993 / 1.0
Validation mAP50 / mAP50-95	0.995 / 0.964
Vela NPU coverage	100% (0 CPU-fallback ops, no quantisation warnings)
Offline confidence test	60 / 60 correct class detections vs PyTorch ground truth
On-device timing	preprocess ≈ 7 ms, inference ≈ 88 ms, postprocess ≈ 0–1 ms
Weights	best.pt (training/export only; never used for PC inference)

Data limitation

The current training data is limited to single-object reference images. Multi-object detection already works in the SenseCraft tests (Figure 3), but a full field test in the real competition scene is still needed (Section 12).

6. GV2 Serial Protocol

All PC ↔ GV2 communication uses the SSCMA-Micro AT protocol over USB serial (COM13, 921600 baud).

Item	Detail
Command framing	<COMMAND>\r
Response framing	\r<json>\n
AT+MODEL?	Model slot check. size == 0 means no valid model is loaded, so re-flash
AT+INFO?	Device info and crc16
AT+INVOKE=1,0,X	Runs one inference. X=1: result only (fast). X=0: result plus base64 JPEG image

AT+INVOKE returns two JSON lines: a type 0 operation reply, then a type 1 event reply containing data.bboxes, data.perf and (when X=0) data.image.

```
{
  "type": 1, "name": "INVOKE", "code": 0,
  "data": {
    "count": N, "perf": [7, 88, 0],
    "boxes": [[x, y, w, h, score, target_id], ...],
    "image": "<base64 jpeg>"
  }
}
```

`boxes == []` with `code == 0` means the inference ran but nothing passed the threshold. This is a different failure from `size == 0` (no model loaded).

Safety rule: no recursive invoke

An earlier SSCMA callback design invoked from inside `on_connect`, causing a recursive-invoke bug. The current flow is single-threaded: open port → 0.3 s settle → loop { invoke → read → parse }.

7. Software Tools

File	Use it when
<code>gv2_serial_test.py</code>	First step: minimal diagnostic (is a model loaded? does invoke return anything?)
<code>gv2_detect.py</code>	Live video, boxes and X/Y/W/H/CX/CY in the terminal
<code>calibration.py</code>	Mapping camera pixels to robot coordinates
<code>gv2_to_mega_bridge.py</code>	Forwarding detections to the Mega during development
<code>README_VSCODE.md</code>	Running everything from VS Code

7.1 gv2_serial_test.py

Runs one command at a time and prints the raw device reply, so the firmware's answer can be read directly.

```
python gv2_serial_test.py --port COM13 --cmd model
python gv2_serial_test.py --port COM13 --cmd info
python gv2_serial_test.py --port COM13 --cmd invoke
python gv2_serial_test.py --port COM13 --cmd raw --raw-command "AT+ID?"
```

7.2 gv2_detect.py (main viewer)

Press Run in VS Code; defaults come from the config block at the top (`COM_PORT`, `BAUD_RATE`, `CONFIDENCE_THRESHOLD`). Flow:

1. Open the port, wait 0.3 s, print "GV2 CONNECTED".
2. `invoke_once()` sends `AT+INVOKE=1,0,0` (with image).
3. `decode_image_field()` decodes the base64 JPEG to an OpenCV frame.
4. `to_detections()` filters by threshold and converts boxes to `Detection` objects (centre/top-left handled).
5. `print_detections()` prints class, confidence, X, Y, W, H, CX, CY.
6. `draw_detections()` draws boxes, labels, centre dots and the "GV2 CAMERA | Objects: N" header.
Press q to quit.

Option	Purpose
<code>--port</code> , <code>--baud</code>	COM port and baud rate
<code>--threshold</code>	Confidence cut-off (0–1); try 0.3 / 0.4 / 0.5 / 0.6 on hardware
<code>--result-only</code>	Skip image transfer for lower latency (no preview)

Option	Purpose
<code>--once</code>	Single invoke, then exit
<code>--interval</code>	Seconds between invokes in continuous mode

7.3 calibration.py (pixel → robot)

The camera rides on the arm, so no fixed-camera geometry is assumed. The script fits a **planar homography** from at least four measured pairs of pixel centre (CX, CY) and known robot (X, Y):

$$[rx, ry, 1]^T \sim H * [px, py, 1]^T$$

1. **Collect:** `python calibration.py collect --out calibration_points.json` (minimum 4 points).
2. **Fit:** `python calibration.py fit --in calibration_points.json --out homography.json`.
Mean and max reprojection error are printed; a warning appears above 5 units.
3. **Use:** `PixelToRobot.load("homography.json").map(cx, cy)` returns (rx, ry).

Limitations

(a) Valid only for one plane/height; recalibrate if the pick height changes. (b) Valid only for the fixed arm "look pose" used during calibration; return to that pose before every detection, or calibrate per pose. (c) A full hand-eye calibration needs the arm's forward kinematics, which are not available yet.

With OpenCV installed, `cv2.findHomography` is used (RANSAC for 5+ points); otherwise a plain DLT/SVD fallback. Nothing is hard-coded: every number in `homography.json` comes from measured points.

7.4 gv2_to_mega_bridge.py (development bridge)

Reads detections from GV2 and sends the Mega one text line per detection. Integers are used (confidence × 100, centre × 10) so the Mega needs no float parsing. The Mega resets when its serial port opens, hence the 2 s wait.

```
D,<class_id>,<confidence_x100>,<x>,<y>,<w>,<h>,<cx_x10>,<cy_x10>\n
python gv2_to_mega_bridge.py --gv2-port COM13 --gv2-baud 921600 \
    --mega-port COM7 --mega-baud 115200 --threshold 0.5
```

For the competition, the same job is done by the XIAO on-device bridge with Mega firmware (delivered, see Section 13), so no PC is needed.

8. Setup and Run Guide

8.1 One-time setup

1. Open the project folder in VS Code.
2. Run `pip install pyserial opencv-python numpy`.
3. Find the GV2 COM port in Device Manager → Ports (COM & LPT).

4. Set `COM_PORT` in `gv2_detect.py` and `gv2_serial_test.py`.

8.2 Normal workflow

1. **Model check:** `python gv2_serial_test.py --cmd model` should report "Model present".
2. **Invoke check:** `python gv2_serial_test.py --cmd invoke` should return boxes.
3. **Live view:** `python gv2_detect.py` shows video and coordinates.
4. **Calibration:** place the object at known positions, run `calibration.py collect`, then `fit`.
5. **Mega test:** run `gv2_to_mega_bridge.py`.

One program per COM port

The SenseCraft browser tab, `gv2_detect.py` and the bridge cannot all hold COM13 at once. Close one before starting another.

9. Issues and Resolutions

Issue	Root cause	Resolution	Status
Empty detections (<code>boxes: []</code>)	YOLO26 metadata not supported by GV2/SenseCraft firmware (confirmed by Seeed)	Rebuilt on YOLOv8n	Resolved
No usable labels	Bulk image folder unlabelled	Recovered <code>annotations.json</code> from 4 SenseCraft exports; merged to 400-image set	Resolved
Incompatible export (NCHW, float32 I/O)	New Ultralytics default backend	Used verified older export path (INT8, NHWC)	Resolved
<code>size == 0</code> on <code>AT+MODEL?</code>	No valid model in the slot	Re-flash via SenseCraft; <code>--cmd model</code> now checks it	Resolved
Boxes offset from objects	Coordinate space + (x, y) is centre, not top-left	<code>BOX_XY_IS_CENTER</code> and frame-size check	Fixed, verified
Recursive invoke	Invoke called inside <code>on_connect</code>	Single-threaded read loop	Fixed by design
Labels overlap in small window	~300 px window with 9–10 objects	Enlarge window / smaller font / higher threshold	Cosmetic, open

10. Current Project Status

Component	Status	Verification
AI model (YOLOv8n, 4-class)	Hardware-validated	Confirmed on real GV2 via SenseCraft logger

Component	Status	Verification
Vela / NPU compilation	Validated	100% NPU, 0 CPU-fallback ops, no quantisation warnings
Offline confidence test	Passed	60/60 held-out images match PyTorch ground truth
GV2 → PC detection script	Hardware-validated	Live detection and box/centre alignment confirmed
Coordinate mapping (box and centre)	Fixed and verified	Scale and centre-vs-corner issues root-caused and confirmed
XIAO → Mega bridge	Delivered, not hardware-tested	Code provided; physical wiring test pending
Pixel → robot calibration	Delivered, not yet run	<code>calibration.py</code> ready; real points needed
Arm / motor control logic	Not implemented	Needs arm kinematics / driver specification

11. Open Items to Verify

1. Bridge uses the wrong box convention

`gv2_detect.py` treats (x, y) as the box centre, but `gv2_to_mega_bridge.py` still computes $cx = x + w/2$, $cy = y + h/2$. The Mega would receive a centre that is off by half a box. Fix: use $cx = x$, $cy = y$ and send the top-left as $x - w/2$, $y - h/2$.

2. Confirm the scaling step in `gv2_detect.py`

The project notes describe rescaling the model's 192×192 output to the 240×240 preview. In the `gv2_detect.py` copy supplied for this document, that scaling is commented out (`scale_x = scale_y = 1.0`). Also, the SenseCraft log contains values such as $y = 235$ and $x = 201$, $w = 31$, which exceed 192; this suggests the device already reports boxes in 240×240 space. Before calibrating, confirm which version is final: print the largest x/y values from a live frame. If they go beyond 192, no rescale should be applied. Calibration and runtime must use exactly the same convention.

3. Stale comment

The `MODEL_INPUT_SIZE` comment still describes boxes as 192-space; update it once item 2 is settled.

12. Remaining Work

- Pixel-to-robot calibration:** collect real reference points (at least 4, ideally 6–9 across the table) at one fixed arm look-pose and run `calibration.py`.
- XIAO → Mega hardware bridge:** wire and test it so detections reach the Mega without a PC, as required at competition time.
- Arm / motor integration:** implement pick, sort and place motion once arm kinematics and driver details are available.

4. **End-to-end field test:** validate the full pipeline in the real competition scene (multiple objects, real backgrounds, venue lighting).
5. **Fix the items in Section 11** before calibration.

13. Deliverables

- Trained model files: `best.pt`, `best_full_integer_quant_vela.tflite` (NPU-ready), a pre-Vela debug copy, `labels.txt` and full metadata.
- Dataset used for training, in YOLO format, with the reproducibility script.
- PC-side Python tools: live detection viewer, serial diagnostics, pixel-to-robot calibration and the GV2-to-Mega development bridge.
- Arduino firmware for the Mega (final receiver) and the XIAO (on-device bridge).
- Documentation: deployment guide, hardware connection guide, serial protocol reference and troubleshooting checklist.

14. Quick Reference

Common commands

```
# diagnostic
python gv2_serial_test.py --port COM13 --cmd model
python gv2_serial_test.py --port COM13 --cmd invoke

# live detection
python gv2_detect.py --port COM13 --baud 921600 --threshold 0.5
python gv2_detect.py --once
python gv2_detect.py --result-only      # fast, no video

# calibration
python calibration.py collect --out calibration_points.json
python calibration.py fit --in calibration_points.json --out homography.json

# bridge
python gv2_to_mega_bridge.py --gv2-port COM13 --mega-port COM7
```

Troubleshooting cheat-sheet

Symptom	Likely cause	Action
"FAILED to open COM13"	Port busy (SenseCraft tab or another script) or wrong COM	Close other programs; check Device Manager
"no result from GV2"	Model not loaded, or timeout	Run <code>--cmd model</code>
<code>size == 0</code>	Model slot empty	Re-flash via SenseCraft
<code>boxes == []</code>	Nothing above threshold	Check lighting/angle; lower threshold
Box shifted from object	Centre vs top-left, or scaling	Toggle <code>BOX_XY_IS_CENTER</code> ; see Section 11

Symptom	Likely cause	Action
No video window	OpenCV missing, or <code>--result-only</code>	<code>pip install opencv-python</code>
Wrong centre on the Mega	Bridge assumes top-left	Section 11, item 1

End of document.